

GreenLight – Online Assessment Prep

Java & SQL Practice Questions (Fintech Track)

August 5, 2026

This set covers core Java (OOP, collections, concurrency, exception handling, tricky output-prediction snippets) and SQL (joins, aggregation, window functions, subqueries) at a level suited for a fintech backend OA. Answers with explanations are provided in Part 3.

Part 1: Java (20 Questions)

Q1.

What is the output?

```
public class Test {  
    public static void main(String[] args) {  
        Integer a = 127, b = 127;  
        Integer c = 200, d = 200;  
        System.out.println(a == b);  
        System.out.println(c == d);  
    }  
}
```

- A) true true
- B) true false
- C) false true
- D) false false

Q2.

Predict the output.

```
class Base {  
    static void show() { System.out.println("Base static"); }  
    void display() { System.out.println("Base display"); }  
}  
class Derived extends Base {  
    static void show() { System.out.println("Derived static"); }  
    void display() { System.out.println("Derived display"); }  
}  
public class Test {  
    public static void main(String[] args) {  
        Base b = new Derived();  
        b.show();  
    }  
}
```

```
        b.display();  
    }  
}
```

- A) Base static
Base display
- B) Base static
Derived display
- C) Derived static
Derived display
- D) Derived static
Base display

Q3.

What does this print?

```
public class Test {  
    public static void main(String[] args) {  
        String s1 = "hello";  
        String s2 = "hello";  
        String s3 = new String("hello");  
        System.out.println(s1 == s2);  
        System.out.println(s1 == s3);  
        System.out.println(s1.equals(s3));  
    }  
}
```

- A) true, true, true
- B) true, false, true
- C) false, false, true
- D) true, false, false

Q4.

Which collection would you use for a thread-safe, high-throughput read scenario with occasional writes (e.g. caching live FX rates)?

- A) HashMap
- B) Hashtable
- C) ConcurrentHashMap
- D) Collections.synchronizedMap(new HashMap<>())

Q5.

What is the output?

```
public class Test {  
    public static void main(String[] args) {  
        try {
```

```
        System.out.println("Try");
        throw new RuntimeException("A");
    } finally {
        System.out.println("Finally");
    }
}
```

- A) Try
Finally
(then exception propagates)
- B) Try
(exception propagates, Finally skipped)
- C) Compilation error
- D) Try
Finally
Program exits normally

Q6.

In a financial application, why is `BigDecimal` preferred over `double` for currency calculations?

- A) `BigDecimal` is faster at arithmetic
- B) `double` cannot store negative numbers
- C) `double` uses binary floating-point representation causing rounding errors
- D) `BigDecimal` uses less memory

Q7.

What's the output?

```
public class Test {
    public static void main(String[] args) {
        int x = 5;
        int y = x++ + ++x;
        System.out.println(x + " " + y);
    }
}
```

- A) 7 11
- B) 7 12
- C) 6 11
- D) 7 10

Q8.

What's wrong with this singleton in a multi-threaded environment?

```
public class ConfigManager {
    private static ConfigManager instance;
```

```
public static ConfigManager getInstance() {  
    if (instance == null) {  
        instance = new ConfigManager();  
    }  
    return instance;  
}  
}
```

- A) Nothing, it's correct
- B) Two threads could create two separate instances (race condition)
- C) It throws NullPointerException
- D) It causes a memory leak

Q9.

What is printed?

```
public class Test {  
    public static void main(String[] args) {  
        List<Integer> list = new ArrayList<>(List.of(1, 2, 3));  
        for (Integer i : list) {  
            if (i == 2) list.remove(i);  
        }  
        System.out.println(list);  
    }  
}
```

1, 3

1, 2, 3

- A) Throws ConcurrentModificationException
- B) Throws IndexOutOfBoundsException

Q10.

Which functional interface fits a lambda taking an Account and returning boolean (e.g. filtering active accounts)?

- A) Function<Account, Boolean>
- B) Consumer<Account>
- C) Predicate<Account>
- D) Supplier<Boolean>

Q11.

Tricky output – what gets printed?

```
public class Test {  
    public static void main(String[] args) {  
        StringBuilder sb1 = new StringBuilder("abc");  
        StringBuilder sb2 = sb1.append("def");  
    }  
}
```

```
        sb1 = new StringBuilder("xyz");  
        System.out.println(sb2);  
    }  
}
```

- A) xyz
- B) abcdef
- C) abc
- D) xyzdef

Q12.

What happens when this code runs?

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int[] arr = new int[3];  
            System.out.println(arr[5]);  
        } catch (Exception e) {  
            System.out.println("Caught: " + e.getClass().getSimpleName()  
                                ());  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Specific catch");  
        }  
    }  
}
```

- A) Prints "Specific catch"
- B) Prints "Caught: ArrayIndexOutOfBoundsException"
- C) Compilation error – unreachable catch block
- D) Runtime crash, nothing prints

Q13.

What is the output?

```
public class Test {  
    static int counter = 0;  
    static int increment() {  
        return counter++;  
    }  
    public static void main(String[] args) {  
        int a = increment() + increment() + increment();  
        System.out.println(a + " " + counter);  
    }  
}
```

- A) 3 3
- B) 0 3
- C) 6 3

D) 3 0

Q14.

Which statement about `HashMap` vs `TreeMap` vs `LinkedHashMap` is correct?

- A) `HashMap` maintains insertion order
- B) `TreeMap` sorts keys naturally (or via `Comparator`) and has $O(\log n)$ get/put
- C) `LinkedHashMap` sorts by key value
- D) `TreeMap` has $O(1)$ average get/put like `HashMap`

Q15.

Tricky – what's the output?

```
public class Test {
    public static void main(String[] args) {
        String result = "";
        for (int i = 0; i < 3; i++) {
            try {
                if (i == 1) throw new RuntimeException();
                result += i;
            } catch (RuntimeException e) {
                result += "E";
                continue;
            } finally {
                result += "F";
            }
        }
        System.out.println(result);
    }
}
```

- A) 0FEF2F
- B) 0F1F2F
- C) 0FE2F
- D) 0EF1F2F

Q16.

What is a key difference between `==` and `.equals()` when comparing custom objects, and what must you override if you override `equals()`?

- A) Nothing, `equals()` alone is sufficient
- B) You must also override `hashCode()` to maintain the equals/hashCode contract for hash-based collections
- C) You must override `toString()`
- D) You must make the class `final`

Q17.

What does this print (autoboxing + ternary trap)?

```
public class Test {  
    public static void main(String[] args) {  
        Object result = true ? new Integer(1) : new Double(2.0);  
        System.out.println(result);  
    }  
}
```

- A) 1
- B) 1.0
- C) Compilation error
- D) 2.0

Q18.

In terms of exception hierarchy, which is true?

- A) Exception extends RuntimeException
- B) RuntimeException extends Exception, and both are unchecked
- C) RuntimeException extends Exception; RuntimeException subclasses are unchecked, other Exception subclasses are checked
- D) Error and Exception are unrelated to Throwable

Q19.

What is the output? (Stream / lazy evaluation trap)

```
public class Test {  
    public static void main(String[] args) {  
        List<Integer> nums = List.of(1, 2, 3, 4, 5);  
        Stream<Integer> stream = nums.stream()  
            .filter(n -> {  
                System.out.print("F" + n + " ");  
                return n % 2 == 0;  
            });  
        System.out.println("Stream created, no output yet? ");  
        long count = stream.count();  
        System.out.println("Count: " + count);  
    }  
}
```

- A) Prints F1 F2 F3 F4 F5 immediately when stream is built, then Count: 2
- B) Prints "Stream created..." first, then F1 F2 F3 F4 F5 (or none, JVM-optimized), then Count: 2
- C) Throws IllegalStateException
- D) Nothing prints because streams are immutable

Q20.

Which is true about `volatile` in Java?

- A) It guarantees atomicity of compound operations like `i++`
- B) It ensures visibility of changes to a variable across threads but does NOT guarantee atomicity
- C) It is equivalent to `synchronized`
- D) It can only be applied to primitive types

Q21.

Tricky output – inheritance + constructors:

```
class A {
    A() { System.out.println("A"); init(); }
    void init() { System.out.println("A init"); }
}
class B extends A {
    int x = 10;
    B() { System.out.println("B, x=" + x); }
    void init() { System.out.println("B init, x=" + x); }
}
public class Test {
    public static void main(String[] args) {
        new B();
    }
}
```

- A) A
A init
B, x=10
- B) A
B init, x=0
B, x=10
- C) A
B init, x=10
B, x=10
- D) B, x=10
A
A init

Part 2: SQL (20 Questions)

Schema used across questions: Accounts(account_id, customer_id, balance), Transactions(txn_id, account_id, amount, txn_date, type), Customers(customer_id, name, signup_date).

Q1.

Why does this query use `HAVING` instead of `WHERE` for the sum condition?

```
SELECT account_id, SUM(amount) AS total
FROM Transactions
WHERE txn_date >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY account_id
```

```
HAVING SUM(amount) > 10000;
```

- A) HAVING is faster than WHERE
- B) WHERE filters rows before aggregation; aggregate functions cannot be used in WHERE
- C) They are interchangeable here
- D) HAVING must always follow GROUP BY regardless of context

Q2.

What's the difference between INNER JOIN and LEFT JOIN here?

```
SELECT c.name, l.amount
FROM Customers c
LEFT JOIN Loans l ON c.customer_id = l.customer_id;
```

- A) Both return identical results
- B) LEFT JOIN returns all customers (NULLs where no loan exists); INNER JOIN returns only customers with a matching loan
- C) LEFT JOIN returns only customers without loans
- D) INNER JOIN returns all rows from both tables

Q3.

Which finds the second-highest transaction amount *per account* (portable, no LIMIT/OFFSET)?

- A) SELECT MAX(amount) FROM Transactions WHERE amount < (SELECT MAX(amount) FROM Transactions)
- B) SELECT amount FROM Transactions ORDER BY amount DESC LIMIT 1 OFFSET 1
- C) Use DENSE_RANK() OVER (PARTITION BY account_id ORDER BY amount DESC) and filter rank = 2
- D) Use COUNT(*) grouped by amount

Q4.

What does this query return?

```
SELECT account_id
FROM Transactions
GROUP BY account_id
HAVING COUNT(DISTINCT txn_date) = COUNT(txn_date);
```

- A) Accounts with no transactions
- B) Accounts where no two transactions happened on the same date
- C) Accounts with duplicate transactions
- D) This is a syntax error

Q5.

Why add a composite index on `Transactions(account_id, txn_date)` rather than two separate single-column indexes?

- A) Composite indexes always use less storage
- B) It efficiently supports queries filtering on `account_id` and sorting/ranging on `txn_date` in a single index scan
- C) It prevents duplicate transactions
- D) SQL requires composite indexes for foreign keys

Q6.

Tricky – what does this query compute?

```
SELECT a.account_id,
       a.balance,
       COALESCE(SUM(t.amount), 0) AS txn_total
FROM Accounts a
LEFT JOIN Transactions t ON a.account_id = t.account_id
GROUP BY a.account_id, a.balance
HAVING a.balance <> COALESCE(SUM(t.amount), 0);
```

- A) Accounts whose stored balance does NOT match the sum of their transaction amounts (potential data inconsistency)
- B) Accounts with zero transactions
- C) All accounts, unfiltered
- D) Syntax error – cannot reference `a.balance` in `HAVING`

Q7.

What's the key difference between `UNION` and `UNION ALL`?

- A) No difference, both remove duplicates
- B) `UNION` removes duplicate rows (extra sort/dedup cost); `UNION ALL` keeps all rows including duplicates and is faster
- C) `UNION ALL` only works with numeric columns
- D) `UNION` preserves order, `UNION ALL` does not

Q8.

Given `Transactions(txn_id, account_id, amount, txn_date)`, which correctly gets a running (cumulative) balance per account ordered by date?

- A) `SELECT account_id, amount, SUM(amount) OVER (PARTITION BY account_id ORDER BY txn_date) AS running_total FROM Transactions`
- B) `SELECT account_id, SUM(amount) FROM Transactions GROUP BY account_id`
- C) `SELECT account_id, amount, AVG(amount) OVER () FROM Transactions`
- D) Running totals aren't possible in standard SQL

Q9.

Tricky NULL handling – what does this return?

```
SELECT COUNT(*), COUNT(account_id), COUNT(closed_date)
FROM Accounts;
-- Assume 100 rows total, closed_date is NULL for 80 open accounts
```

- A) 100, 100, 100
- B) 100, 100, 20
- C) 100, 80, 20
- D) NULL, 100, 20

Q10.

What's wrong (or is anything wrong) with this query?

```
SELECT account_id, amount
FROM Transactions
WHERE amount = NULL;
```

- A) Returns rows where amount is NULL
- B) Returns zero rows always – must use IS NULL instead of = NULL since NULL comparisons yield UNKNOWN
- C) Throws a syntax error
- D) Returns all rows

Q11.

Which correctly finds customers who have NEVER made a transaction, using a correlated sub-query?

- A) SELECT * FROM Customers c WHERE EXISTS (SELECT 1 FROM Transactions t JOIN Accounts a ON t.account_id=a.account_id WHERE a.customer_id = c.customer_id)
- B) SELECT * FROM Customers c WHERE NOT EXISTS (SELECT 1 FROM Transactions t JOIN Accounts a ON t.account_id=a.account_id WHERE a.customer_id = c.customer_id)
- C) SELECT * FROM Customers WHERE customer_id IN (SELECT customer_id FROM Transactions)
- D) SELECT * FROM Customers, Transactions

Q12.

What is the effect of this query, and what's a subtle bug risk?

```
SELECT customer_id, name
FROM Customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM Accounts WHERE customer_id IS NOT NULL
);
```

If the inner subquery's `customer_id` column could still contain a `NULL` (despite the filter being slightly different, e.g. missing the filter), what happens to `NOT IN`?

- A) `NULL`s are ignored automatically, no issue
- B) If the subquery result set contains even one `NULL`, the entire `NOT IN` returns no rows (due to three-valued logic) – a classic SQL gotcha
- C) It throws a runtime error
- D) `NOT IN` behaves identically to `NOT EXISTS` in all cases

Q13.

Which window function assigns a unique sequential rank even for ties (no gaps, no duplicates), unlike `RANK()`?

- A) `RANK()`
- B) `DENSE_RANK()`
- C) `ROW_NUMBER()`
- D) `NTILE()`

Q14.

Given `RANK()` vs `DENSE_RANK()` vs `ROW_NUMBER()` over values `[100, 100, 90, 80]`, what ranks does `RANK()` assign?

- A) 1, 2, 3, 4
- B) 1, 1, 2, 3
- C) 1, 1, 3, 4
- D) 1, 2, 2, 3

Q15.

What does this query do, and is it efficient?

```
SELECT *  
FROM Transactions  
WHERE YEAR(txn_date) = 2026 AND MONTH(txn_date) = 6;
```

- A) It's the most efficient way to filter by month
- B) Wrapping `txn_date` in functions prevents the query optimizer from using an index on `txn_date`; a range predicate (`txn_date >= '2026-06-01' AND txn_date < '2026-07-01'`) is more sargable
- C) It causes a syntax error
- D) `YEAR()` and `MONTH()` don't exist in SQL

Q16.

What's the difference between `DELETE`, `TRUNCATE`, and `DROP`?

- A) They are all identical
- B) **DELETE** removes rows (can be filtered, logged, rollback-able), **TRUNCATE** removes all rows fast (minimal logging, resets identity, hard to roll back), **DROP** removes the entire table structure
- C) **TRUNCATE** only works on views
- D) **DROP** only removes data, not the table

Q17.

Tricky self-join – what does this find?

```
SELECT t1.account_id, t1.amount, t1.txn_date
FROM Transactions t1
JOIN Transactions t2
  ON t1.account_id = t2.account_id
  AND t1.amount = t2.amount
  AND t1.txn_id <> t2.txn_id;
```

- A) All transactions
- B) Transactions that share the same account and amount as at least one other transaction (potential duplicate detection)
- C) Transactions with unique amounts only
- D) A cartesian product with no meaningful filter

Q18.

Which best explains **GROUP BY** + multiple aggregate columns ordering pitfalls?

```
SELECT account_id, type, SUM(amount)
FROM Transactions
GROUP BY account_id
ORDER BY SUM(amount) DESC;
```

- A) This runs fine and groups correctly by `account_id` and `type`
- B) This is invalid/undefined in strict SQL modes: `type` appears in **SELECT** but is neither aggregated nor included in **GROUP BY**
- C) **ORDER BY** cannot use aggregate functions
- D) **GROUP BY** requires listing all **SELECT** columns except aggregates in a specific order

Q19.

What's a key advantage of a Common Table Expression (CTE) over a subquery for readability in a multi-step fintech report query?

- A) CTEs always execute faster than subqueries
- B) CTEs let you name and reuse intermediate result sets, improving readability for multi-step logic (e.g., `WITH monthly_totals AS (...) SELECT ... FROM monthly_totals`)
- C) CTEs can only be used with **INSERT** statements
- D) There is no difference; CTEs are just syntactic sugar with no benefit

Q20.

In an ACID-compliant fintech transaction (e.g., transferring money between two accounts), which isolation-related problem does using too low an isolation level (e.g., READ UNCOMMITTED) risk?

- A) Faster deadlocks only, no data issues
- B) Dirty reads – reading uncommitted changes from another transaction that might later be rolled back, leading to incorrect balance calculations
- C) Higher storage usage
- D) It only affects **SELECT** performance, never correctness

Part 3: Answer Key & Explanations – Java

Q1.

Answer: B) true false. `Integer` caches values from -128 to 127 . `a` and `b` reuse the same cached object (`== true`). `200` is outside the cache range, so `c` and `d` are distinct objects (`== false`).

Q2.

Answer: B) Base static / Derived display. Static methods are resolved by the reference type (compile-time, no polymorphism) – so `b.show()` calls `Base.show()`. Instance methods are resolved by the actual object type (runtime polymorphism) – `b.display()` calls `Derived.display()`.

Q3.

Answer: B) true, false, true. String literals are interned in the string pool, so `s1` and `s2` point to the same object. `new String("hello")` explicitly creates a new heap object, so `s1 == s3` is false. `.equals()` compares content, so it's true.

Q4.

Answer: C) ConcurrentHashMap. It uses fine-grained locking / CAS operations for much better concurrent throughput than a fully locked `Hashtable` or `synchronizedMap` wrapper, both of which lock the entire map on every access.

Q5.

Answer: A) Try / Finally / (then exception propagates). `finally` always runs before an exception propagates up the stack, even with no matching `catch`. Since there's no `catch` here, the `RuntimeException` still propagates after "Finally" prints, and (uncaught) crashes the program with a stack trace.

Q6.

Answer: C. Binary floating point can't exactly represent many decimal fractions (e.g. `0.1`), causing rounding errors – unacceptable for money. Always use `BigDecimal` with an explicit scale and `RoundingMode`.

Q7.

Answer: B) 7 12. `x++` returns `5` then `x` becomes `6`. `++x` increments `x` to `7` first, then returns `7`. So `y = 5 + 7 = 12`, and final `x = 7`.

Q8.

Answer: B. Without synchronization, two threads can both pass the `null` check simultaneously and each construct an instance. Fix with double-checked locking + `volatile`, an enum singleton, or a static holder class (initialization-on-demand holder idiom).

Q9.

Answer: C) Throws ConcurrentModificationException. Modifying a list while iterating with a for-each loop invalidates the iterator's `modCount`. Use `Iterator.remove()` or `list.removeIf(i -> i == 2)` instead.

Q10.

Answer: C) Predicate<Account>. `Predicate<T>` models a boolean-valued function of one argument – ideal for `stream().filter(acc -> acc.isActive())`.

Q11.

Answer: B) abcdef. `StringBuilder.append()` mutates and returns the *same* object, so `sb2` refers to that same mutated object ("abcdef"). Reassigning `sb1` to a new `StringBuilder` afterward doesn't affect `sb2`, since they were never the same reference-holding variable, just two references to the original object.

Q12.

Answer: C) Compilation error – unreachable catch block. Java requires more specific exception types to be caught *before* more general ones. Catching `Exception` before `ArrayIndexOutOfBoundsException` makes the second catch block unreachable, which is a compile-time error.

Q13.

Answer: A) 3 3. Post-increment returns the current value then increments. Calls return 0, 1, 2 in left-to-right evaluation order, summing to 3. After three increments, `counter` is 3.

Q14.

Answer: B. `TreeMap` is backed by a red-black tree, keeping keys sorted (natural order or via a `Comparator`), with $O(\log n)$ get/put. `HashMap` has no ordering guarantee; `LinkedHashMap` preserves *insertion* order, not sorted order.

Q15.

Answer: A) 0FEF2F. Walk through: `i=0` → `result+="0"`, finally adds "F" → "0F". `i=1` → exception thrown, catch adds "E" → "0FE", finally still runs adding "F" → "0FEF", then `continue` moves to next iteration (finally always runs even before `continue` takes effect). `i=2` → `result+="2"` → "0FEF2", finally adds "F" → "0FEF2F".

Q16.

Answer: B. If you override `equals()` without overriding `hashCode()`, you break the contract that equal objects must have equal hash codes – causing subtle bugs in `HashMap`, `HashSet`, etc. (e.g. duplicate "equal" entries, failed lookups).

Q17.

Answer: B) 1.0. This is a classic ternary-operator numeric promotion trap: when a ternary's two branches are `Integer` and `Double` (boxed numeric types), Java unboxes both and promotes to the wider type `double` *even if the condition would pick the `Integer` branch*. So the result is `1.0`, not `1`.

Q18.

Answer: C. `RuntimeException` extends `Exception`, which extends `Throwable`. `RuntimeException` (and its subclasses) are *unchecked*; other `Exception` subclasses (like `IOException`) are *checked* and must be declared or caught. `Error` also extends `Throwable`, separately from `Exception`.

Q19.

Answer: B. Streams are lazy – intermediate operations like `filter` don't execute until a terminal operation (`count()`, `collect()`, etc.) is invoked. So "Stream created..." prints first. Note: for `count()` specifically, some JVMs may even skip calling the filter predicate entirely if it can compute size without traversal in certain cases – but conceptually, nothing runs before the terminal op is called.

Q20.

Answer: B. `volatile` guarantees visibility (all threads see the latest write) and prevents instruction reordering around it, but it does *not* make compound operations like `i++` atomic – that still requires `synchronized` or `AtomicInteger`.

Q21.

Answer: B) "A" / "B init, x=0" / "B, x=10". The superclass constructor `A()` runs first, printing "A", then calls `init()`. Because `init()` is overridden, dynamic dispatch invokes `B.init()` – but `B`'s field initializer (`x = 10`) hasn't run yet at this point (field initializers run after `super()` returns but before `B`'s own constructor body executes), so `x` is still its default value `0`, printing "B init, x=0". After `A()` finishes, `B`'s field initializer sets `x=10`, then `B()`'s body prints "B, x=10". This is a classic Java gotcha: never call overridable methods from a constructor.

Part 3 (contd.): Answer Key & Explanations – SQL

Q1.

Answer: B. `WHERE` filters individual rows *before* `GROUP BY`/aggregation happens, so aggregate functions like `SUM()` aren't valid there. `HAVING` filters *after* grouping, on the aggregated result.

Q2.

Answer: B. `LEFT JOIN` preserves every row from the left table (`Customers`), filling unmatched right-side columns with `NULL`. `INNER JOIN` only returns rows with a match in both tables.

Q3.

Answer: C. `LIMIT/OFFSET` isn't standard SQL (fails on SQL Server without `OFFSET...FETCH`) and doesn't naturally partition per account. `DENSE_RANK() OVER (PARTITION BY account_id ORDER BY amount DESC)` filtered to `rank = 2` is the standard, portable, per-group approach.

Q4.

Answer: B. `COUNT(DISTINCT txn_date)` counts unique dates; `COUNT(txn_date)` counts all rows. Equal counts mean every transaction happened on a distinct date – i.e., no two transactions share a date for that account.

Q5.

Answer: B. A composite index optimizes queries filtering on the leading column and ranging/sorting on the trailing column – exactly `WHERE account_id = ? ORDER BY txn_date`. Two separate single-column indexes would require an index intersection, typically less efficient than one composite index scan.

Q6.

Answer: A. This finds accounts where the stored `balance` field disagrees with the sum of that account's transactions – useful for detecting data drift/inconsistency between a cached balance column and the transaction ledger (very common fintech reconciliation check). `HAVING` can reference grouped/non-aggregated columns from the `GROUP BY` list, like `a.balance` here, so it's valid.

Q7.

Answer: B. `UNION` deduplicates (implicit sort/hash for comparison, more expensive); `UNION ALL` keeps every row including duplicates and skips the dedup step, so it's faster when duplicates are acceptable or known not to exist.

Q8.

Answer: A. `SUM(amount) OVER (PARTITION BY account_id ORDER BY txn_date)` computes a running/cumulative total per account as rows are ordered by date – exactly what a running balance needs. Plain `GROUP BY` collapses rows into one total per account instead of a per-row running value.

Q9.

Answer: C. `COUNT(*)` counts all rows regardless of NULLs = 100. `COUNT(account_id)` counts non-null values in that column = 100 (assuming it's always populated). `COUNT(closed_date)` only counts non-null values = 20 (100 - 80 open/NULL accounts).

Q10.

Answer: B. In SQL, NULL represents "unknown," so `= NULL` always evaluates to UNKNOWN (never true), returning zero rows regardless of actual NULL values. You must use `IS NULL` (or `IS NOT NULL`) for NULL checks.

Q11.

Answer: B. `NOT EXISTS` with a correlated subquery correctly finds customers with zero matching transaction rows. Option A (plain `EXISTS`) would find customers who *have* transacted – the opposite of what's asked.

Q12.

Answer: B. This is a classic SQL gotcha: due to three-valued logic (TRUE/FALSE/UNKNOWN), if the subquery used by `NOT IN` returns *any* NULL in its result set, the entire `NOT IN` comparison becomes UNKNOWN for every row, so the query silently returns zero rows. `NOT EXISTS` avoids this problem entirely and is generally safer.

Q13.

Answer: C) ROW_NUMBER(). `ROW_NUMBER()` always assigns strictly sequential, unique integers (1, 2, 3, 4...) even for tied values. `RANK()` leaves gaps after ties; `DENSE_RANK()` doesn't leave gaps but does repeat ranks for ties.

Q14.

Answer: C) 1, 1, 3, 4. `RANK()` gives tied rows the same rank, then *skips* the next rank(s) equal to the number of ties. Two rows tied at rank 1 means the next distinct value gets rank 3 (not 2), and the last gets rank 4.

Q15.

Answer: B. Wrapping an indexed column in a function (`YEAR(txn_date)`, `MONTH(txn_date)`) makes the predicate non-sargable – the optimizer generally can't use an index on `txn_date` efficiently and may need a full scan. A direct range comparison on the raw column allows index usage.

Q16.

Answer: B. `DELETE` removes rows (can be filtered with `WHERE`, is logged row-by-row, and is transactionally rollback-able). `TRUNCATE` deallocates all rows at once with minimal logging (fast, often resets auto-increment, harder/impossible to roll back in some DBs). `DROP` removes the table structure entirely, including schema and data.

Q17.

Answer: B. The self-join matches each transaction to any *other* transaction (`t1.txn_id <> t2.txn_id`) on the same account with the exact same amount – a common pattern for flagging potential duplicate or suspicious repeated transactions.

Q18.

Answer: B. In standard SQL (and strict modes like MySQL's `ONLY_FULL_GROUP_BY`), every non-aggregated column in `SELECT` must appear in `GROUP BY`. Here `type` is selected but not grouped or aggregated – which is invalid/undefined behavior in strict SQL engines, even though some lenient configurations may silently pick an arbitrary value.

Q19.

Answer: B. CTEs (`WITH clause_name AS (...)`) let you break a complex query into named, readable, reusable steps – especially valuable for multi-stage fintech reports (e.g., compute monthly totals, then rank them, then filter). They don't inherently guarantee better performance than an equivalent subquery; the main benefit is clarity and (in some DBs) reusability within the same query.

Q20.

Answer: B. `READ UNCOMMITTED` allows dirty reads – a transaction can read another transaction's uncommitted changes. If that other transaction later rolls back, you've based logic (e.g., a balance check before a transfer) on data that never really existed – a serious correctness risk in financial systems. Fintech systems typically require at least `READ COMMITTED`, often `REPEATABLE READ` or `SERIALIZABLE` for critical money-movement operations.

Good luck with your GreenLight OA! Focus especially on: BigDecimal for money, ConcurrentHashMap/thread-safety, exception hierarchy, window functions, and NULL/three-valued-logic gotchas in SQL – these are prime fintech-interview territory.